

Abstraction studio

Revise 360 lesson plan — 60 minutes

Unit	OCR J277 2.1 Algorithms — Paper 2, Computational thinking, algorithms and programming
Lesson	Abstraction studio (lesson 1 of the unit)
Lesson ID	al-l01
Experience ID	al-l01
Plan ID	PLAN-AL-L01
Content version	eeb77f180255
Website	https://revise360.co.uk/experience.html?id=al-l01

Big question, outcomes and success criteria

Big question — What is abstraction, and why is it needed?

Prerequisites

None within this unit: it is the first lesson. Treat the starter as a baseline check rather than retrieval of something you have taught.

Assessable outcomes

1. **al-l01-o1** — know what is meant by the term abstraction
2. **al-l01-o2** — know some examples of abstraction
3. **al-l01-o3** — understand why abstraction is needed in computational thinking

Vocabulary, resources and preparation

Key vocabulary — Abstraction, Computational thinking

Resources

SLIDES	presentations/AL_L01_AbstractionStudio_Lesson.pptx
WORKSHEET	worksheets/AL_L01_AbstractionStudio_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=al-l01

Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the 360 image is already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on <code>AL_L01_AbstractionStudio_Worksheet.docx</code> is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong.	Correct their own starter.	A correction written, not just a tick.
8-18	Explain and model	The new idea in two or three steps, with the example worked in front of the class.	Listen and answer questions about the example.	They can say why, not only what.
18-21	Set up and settle	Sign-in, launch, and what to write before anyone taps an answer.	Sign in, open the lesson, find Computational thinking.	Everyone is in and on station 1.
21-43	Online: work the stations in order	Pupils read each wall, write the key fact and the challenge, then answer at the badge.	Read each wall, write the key fact and the challenge, then answer at the badge.	The worksheet has writing on it for each of the 6 stations before the badge is tapped.
43-48	Check understanding	The hinge question, hands down.	Answer the hinge question, hands down.	The split across the options.
48-56	Apply	Exam practice on the worksheet, in silence.	Exam practice, in silence.	worksheet, Exam practice a) to c)
56-60	Exit and set up the next lesson	The exit question and the retrieval link to next lesson.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Computational thinking

- Computational thinking is how you approach a problem so a computer can solve it.
- It has three parts: abstraction, decomposition and algorithmic thinking.
- They are used together, not one after the other.

Challenge on this wall: Name the three parts of computational thinking and write one line on each.

2. What abstraction is

- Abstraction means removing detail that does not matter to the problem.
- What is left is a model: simpler than reality, and good enough to work with.
- What counts as unnecessary depends entirely on the problem you are solving.

Challenge on this wall: Write your own definition of abstraction, with an example that is not a map.

3. A worked example

- A school register needs a name, a form group and present or absent.
- It does not need eye colour, shoe size or favourite subject.
- Adding unnecessary data makes the system slower, larger and harder to keep correct.

Challenge on this wall: List four things a school register stores and three it deliberately does not.

4. Models of the world

- A weather forecast is a model: it keeps pressure and temperature, and ignores individual raindrops.
- A satnav keeps roads, junctions and speeds, and ignores what the buildings look like.
- Every model is wrong in some way. A useful model is wrong about things that don't matter.

Challenge on this wall: Choose a program you use and say what it deliberately ignores about the real world.

5. Abstraction in programs

- A variable name like score is an abstraction: it hides which memory address holds the value.
- A subprogram is an abstraction: you call it without needing to know how it works inside.
- Layers of abstraction let one person work on a game without understanding the graphics card.

Challenge on this wall: Explain how calling a subprogram is an example of abstraction.

6. Answering abstraction questions

- Say what is kept and what is removed, and why each decision was made.
- Link the decision to the purpose of the system.
- One mark is usually for an example, so always give one.

Challenge on this wall: Plan a four-mark answer on abstraction in a bus timetable app.

Retrieval and the check for understanding

Starter — Sketch a map that would get a new pupil from the school gate to your classroom. What did you leave out, and why?

Answer space: 4 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (a1-l01-s4-c2), so the wording a pupil sees matches the wording you say.

A flight simulator ignores the weight of the paint on the aircraft. Why is that acceptable?

Correct: It is far too small to change the behaviour being modelled

What a pupil might choose instead

- Paint has no weight
- Simulators cannot model weight
- It would be illegal to include it

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: Abstraction is about judging what makes a difference.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Recognise	10	The answer is on the screen and the pupil picks it: multiple choice, choosing several, matching a pair.
Construct	3	Nothing is offered. The pupil classifies, orders, fills in a table or works a number out, and types or places the answer.

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Computational thinking	2	Computational thinking means...
2	What abstraction is	2	Abstraction is...
3	A worked example	2	In a register, the necessary detail is...
4	Models of the world	2	Models use abstraction because...
5	Abstraction in programs	2	Programs use abstraction when...
6	Answering abstraction questions	2	To answer an abstraction question, I should...
final	Final challenge: keep or cut	1	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
al- l01-x1	State what is meant by abstraction.	2	worksheet, Exam practice a); deck, Exam practice slide

ID	QUESTION	MARKS	WHERE
a1 - l01-x2	Give two examples of abstraction in a computer system.	2	worksheet, Exam practice b); deck, Exam practice slide
a1 - l01-x3	A company is writing a program to plan delivery routes. Explain how abstraction would be used, giving examples of detail kept and removed.	6	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/a1-l01.json`, which is not published with the website.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What are the three parts of computational thinking?** — Input, process and output; Syntax, logic and runtime
- **What does abstraction mean in computational thinking?** — Making a program shorter; Hiding the source code
- **Why is what counts as 'unnecessary' not fixed?** — Because different people prefer different details; Because computers vary
- **Why is storing unnecessary data a problem?** — It makes the screen look busy; It uses more electricity in the CPU
- **A flight simulator ignores the weight of the paint on the aircraft. Wh** — Paint has no weight; Simulators cannot model weight

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- The three parts of computational thinking
- What abstraction is
- Judging necessary and unnecessary detail

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is a1-l02, Breakdown bay. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

